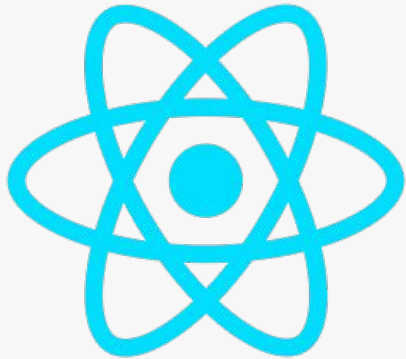




**Newton School
of Technology**

The Ultimate React Course



@newtonschool

Lecture 18: Sharing state between components

JS

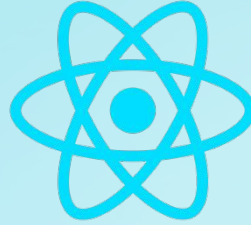
Prerequisites

- React functional components
- Basic Understanding of JSX
- How to launch React Application
- State Management in React



Table of Contents

- Communication Challenge
- Parent → Child Communication
- Child → Parent Communication
- Sibling → Sibling Communication
- Quick Recap



The Communication Challenge

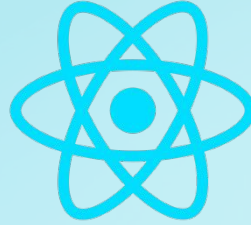
A brief Overview

Why can't child components talk on their own?

If `<LeftPanel />` component updates something (e.g., a filter)... how will `<RightPanel />` component (e.g., the product list) know?

- React enforces **unidirectional (top → down) data flow**.
- Components must communicate using **props** and **state**.
- A common component (**Parent**) must manage and share data to enable syncing.

```
<Parent>  
  <LeftPanel />  
  <RightPanel />  
</Parent>
```



Parent → Child Communication

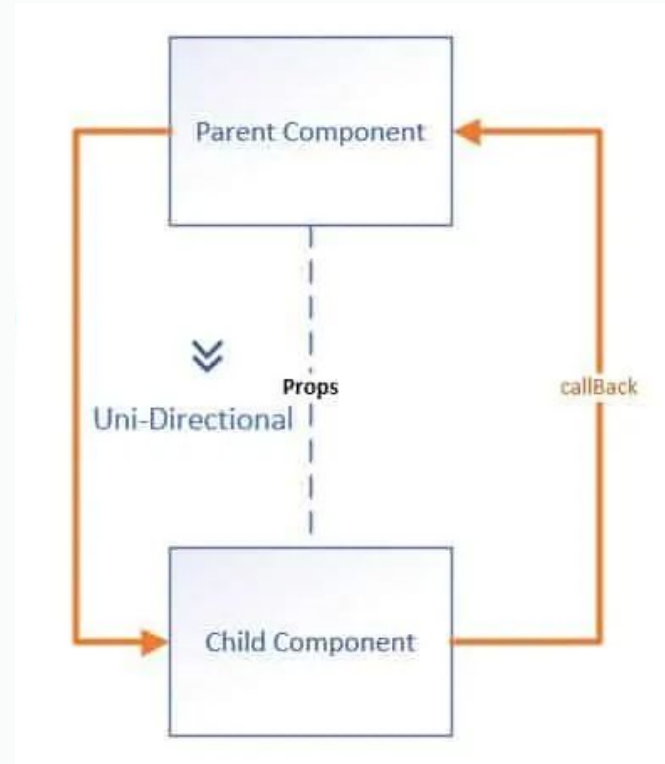
A brief Overview

Why Parent → Child?

UI must reflect data from a **source of truth**, like:

- User name from database
- Theme settings from settings panel
- Product details from API

*Without props?
UI becomes hard-coded, static, useless*



What Are Props?

Props are **read-only** inputs passed from **parent** to **child**.

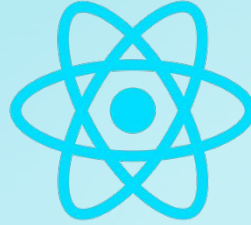
Like **passing a lunch box** to a child → the child **cannot** change its content!

- Used for personalization
- Follow **top-down flow**
- Immutable by child

Child receives data through function parameters

 *Cannot do name = "John" → props are immutable
UI becomes hard-coded, static, useless*

```
function Parent() {  
  const name = "Raghav";  
  return <Child name={name} />;  
}  
  
function Child({ name }) {  
  return <h2>Hello, {name}</h2>;  
}
```

Child → Parent Communication

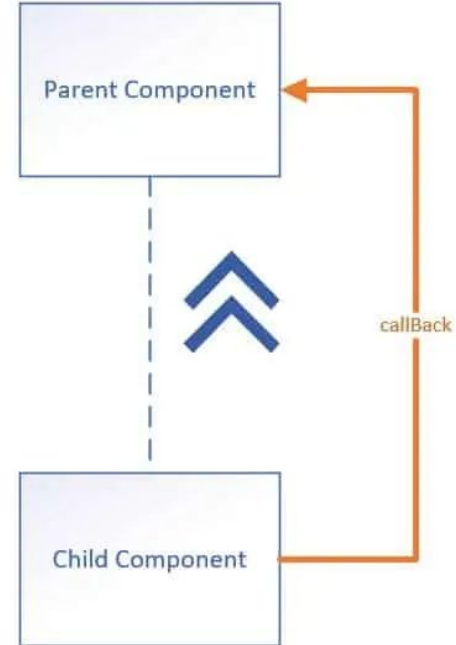
A brief Overview

Why Child → Parent?

Child components trigger actions that the parent needs to respond to:

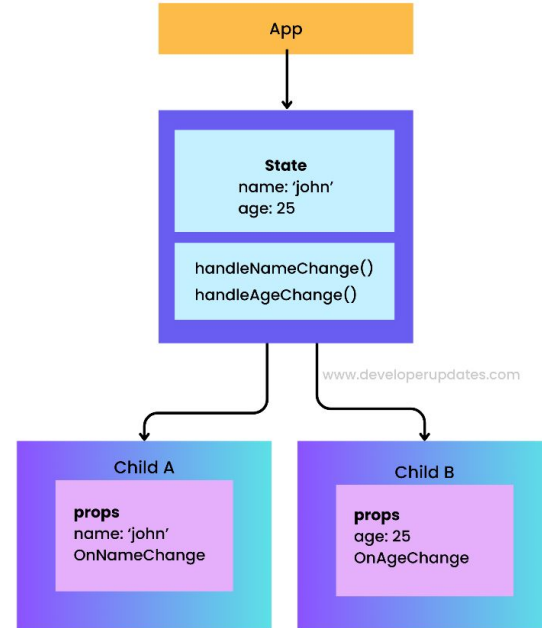
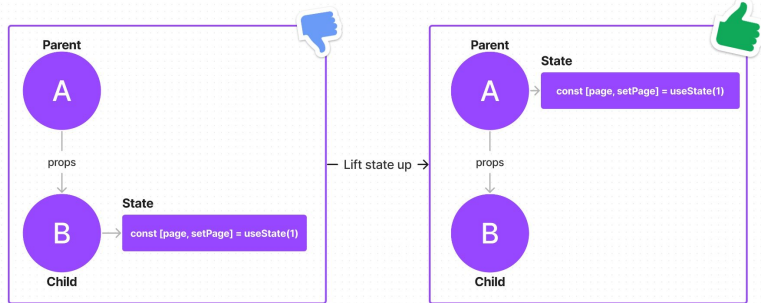
- A **Form submit** button is clicked (✓).
- A **Filter** is applied in a sidebar (✓).
- A **Toggle** button switches dark mode (✓).

Challenge:
But child cannot directly modify parent's state...



What is Lifting State Up?

Move state to the **closest common parent**, then pass **functions** to children so they can *request* changes.



What is Lifting State Up?

Mechanism:

1. **Parent** defines a state (**count**) and a function (**increment**).
2. **Parent** shares the function (**onIncrement**) with the child via a prop.
3. **Child** calls the prop function → **Parent** updates state.
4. Updated state flows back down to children via props.

```
import { useState } from 'react';

function Parent() {
  const [count, setCount] = useState(0);

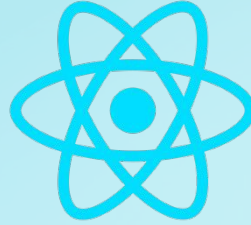
  function increment() { // The Callback Function
    setCount(prev => prev + 1);
  }

  return (
    <>
      <p>Current Count: {count}</p>
      <Child onIncrement={increment} /> {/* Passing the function */}
    </>
  );
}

function Child({ onIncrement }) {
  return <button onClick={onIncrement}>Add to Parent</button>; // Calling the function
}
```

Result:

- ✓ Child triggers parent update
- ✓ UI changes correctly



Sibling → Sibling via Shared Parent

A brief Overview

Why Sibling Communication?

When two components side-by-side need to be synchronized, example:

UI Feature

Search bar + Results

Filters + Products

Login modal + Navbar

Tabs + Panels

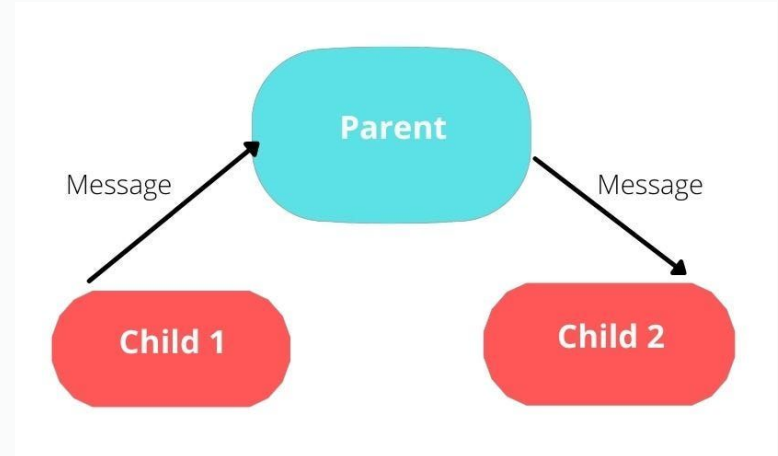
Shared State Example

Search term

Selected filters

Auth status

Active tab



*Direct child → child passing ❌
Hierarchy must be respected ✓*

How Siblings Share State?

Rule:

Siblings must communicate through the **nearest common parent**.

Mechanism:

1. **State** lives in the common **Parent**.
2. **Parent** passes:
 - State (data) to both siblings as **props**.
 - State Setter (callback function) to the *modifying* sibling.
3. An update in **Sibling 1** affects the **Parent's state**.
4. The updated state flows down, and **Sibling 2** instantly reflects the change.

```
import { useState } from 'react';

function Parent() {
  const [text, setText] = useState(""); // Shared State

  return (
    <div>
      <Left text={text} setText={setText} /> {/* Left gets data + setter */}
      <Right text={text} /> {/* Right only needs the data */}
    </div>
  );
}

function Left({ text, setText }) {
  return <input value={text} onChange={(e)=>setText(e.target.value)} />;
}

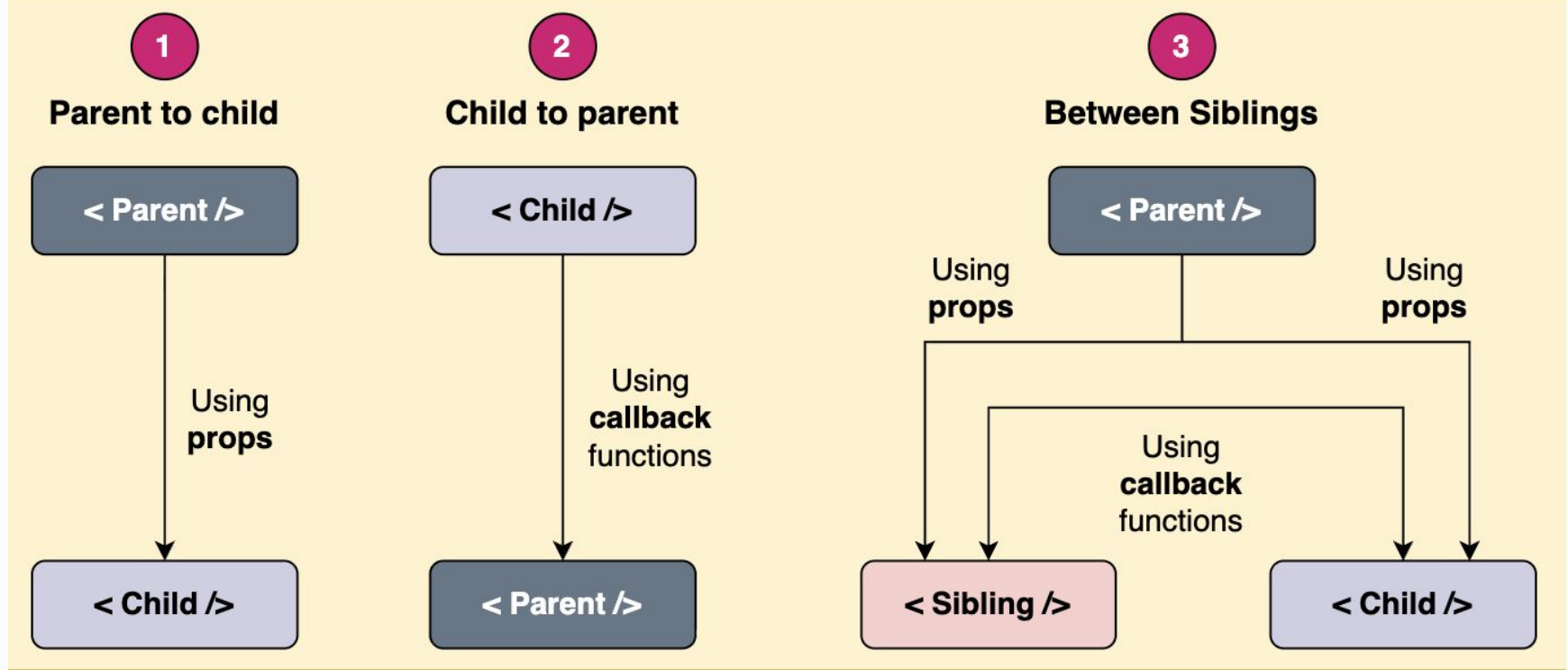
function Right({ text }) {
  return <p>Current: {text}</p>;
}
```

Result:

- ✓ Both fields stay in sync
- ✓ Parent becomes the single source of truth

Quick Recap

Quick Recap:



Quick Recap:

Concept	Direction	Tool	Example
Parent → Child	Downward	Props	Show name in UI
Child → Parent	Upward	Callback props	Child button updates count
Sibling → Sibling	Sideways	Shared parent state	Filter + Results synced

Workshop

In Class Questions

References

1. **MDN React Guide:** Comprehensive and beginner-friendly documentation for React
[https://developer.mozilla.org/en-US/docs/Learn/Tools and testing/Client-side JavaScript frameworks/React getting started](https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started)
2. **Websites and Tutorials:**
 - a.  w3schools: <https://www.w3schools.com/REACT/DEFAULT.ASP>
 - b.  codecademy: <https://www.codecademy.com/learn/react-101>
3. **Other Useful Resources**
 - a.  React GitHub Repository: <https://github.com/facebook/react>
 - b.  React Patterns & Best Practices: <https://reactpatterns.com/>

**Thanks
for
watching!**